Design and Implementation of a Configurable TinyML Acceleration Framework for the NXP MCXN236 Microcontroller

1. Introduction

Tiny Machine Learning (TinyML) enables machine-learning models to operate directly on resource-constrained microcontrollers. Although this removes the need for cloud-based inference, neural-network computation can still impose significant requirements on processor cycles, memory, latency, and energy.

This project proposes a **configurable TinyML acceleration framework for the NXP FRDM-MCXN236 development board**, based on the Arm Cortex-M33 microcontroller. Instead of using an external FPGA or dedicated accelerator chip, the proposed framework will exploit the computational capabilities of the microcontroller through optimized software execution, INT8 quantization, model-specific kernels, and efficient memory management.

The framework will initially target commonly used lightweight models such as **Multi-Layer Perceptrons (MLPs)** and **1D Convolutional Neural Networks (1D CNNs)**. The framework will provide a common execution structure in which the selected model is analyzed and mapped to suitable optimized kernels for efficient inference on the MCXN236.

---

2. Problem Statement

Deploying machine-learning models on microcontrollers involves several challenges:

- Limited computational resources compared with conventional processors.
- High number of multiply-accumulate operations in neural-network layers.
- Limited RAM and Flash memory for storing models and intermediate data.
- Execution latency can become significant for real-time applications.
- Different ML models require different computational operations and memory-access patterns.

- Conventional implementations may not fully exploit the processing capabilities of the target microcontroller.

Therefore, a flexible framework is required that can adapt the execution of different lightweight ML models to the available resources of a microcontroller while maintaining acceptable inference accuracy and improving computational efficiency.

---

3. Proposed Solution

The proposed system is a **model-aware TinyML acceleration framework** designed specifically for the MCXN236.

The framework will consist of:

1. **Model development and training** on a PC using Python-based ML tools.
2. **Transfer learning or conventional training**, depending on the selected model and dataset.
3. **INT8 quantization** to reduce computational and memory requirements.
4. **Model analysis and configuration**, which identifies the layers and operations required by the selected model.
5. **Optimized execution kernels** for operations such as matrix multiplication, multiply-accumulate operations, and 1D convolution.
6. **Memory-efficient execution** to reduce unnecessary data movement and RAM usage.
7. **Embedded inference runtime** running on the Cortex-M33 of the MCXN236.
8. **Performance benchmarking** against a conventional software implementation.

The framework will therefore allow different supported models to use appropriate optimized execution paths rather than relying on one generic implementation.

---

4. System Workflow

The overall workflow will be:

Dataset → Model Training/Transfer Learning → INT8 Quantization → Model Analysis → Kernel Selection/Configuration → MCXN236 Deployment → TinyML Inference → Performance Evaluation

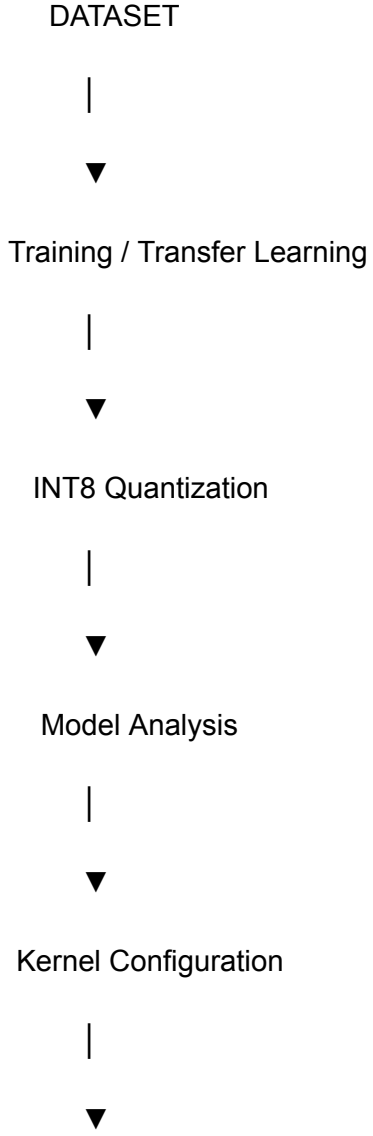
The system can be divided into two stages.

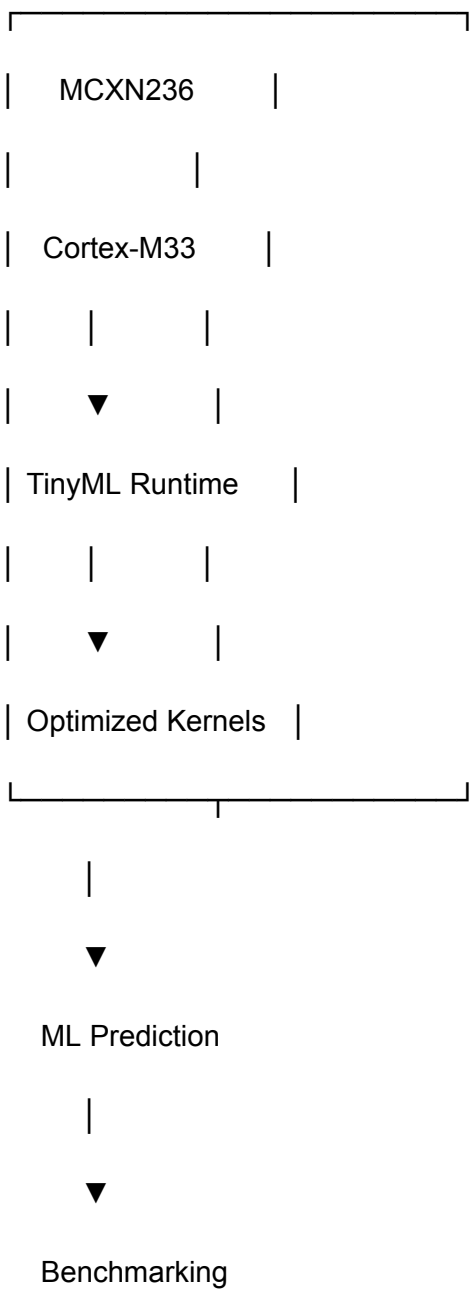
PC-side stage

The ML model is developed and validated using Python tools. After training, the model is quantized to INT8 and converted into a representation suitable for embedded deployment. The framework analyzes the model structure and generates the required configuration and model parameters.

MCXN236-side stage

The converted model is deployed to the microcontroller. The embedded runtime receives the model configuration and executes the required operations using optimized kernels. The Cortex-M33 performs the inference while the framework manages model data, intermediate results, and execution flow.





---

5. Initial ML Models

The framework will initially support two lightweight model classes.

Multi-Layer Perceptron (MLP)

MLPs are primarily composed of fully connected layers and therefore contain a large number of matrix multiplication and multiply-accumulate operations.

The framework will provide optimized execution for these operations.

1D Convolutional Neural Network (1D CNN)

1D CNNs are suitable for sequential and sensor-based data. Their convolution operations can also be implemented using optimized multiply-accumulate kernels.

Supporting both MLP and 1D CNN allows the framework to demonstrate that the proposed execution architecture is not restricted to a single model structure.

---

6. Technologies and Components

The major technologies involved are:

- **NXP FRDM-MCXN236 development board**
- **Arm Cortex-M33**
- C/C++ embedded programming
- Python for ML development and model conversion
- INT8/fixed-point quantization
- MLP and 1D CNN architectures
- Optimized MAC/GEMM/convolution kernels
- Cortex-M33 SIMD/integer processing capabilities
- Memory-efficient buffer management
- MCUXpresso development environment
- Model benchmarking and profiling

---

7. Performance Evaluation

The proposed framework will be compared with a conventional software implementation running on the same MCXN236.

The evaluation will consider:

Parameter	Purpose
Inference latency	Measures execution speed
CPU cycles	Measures computational effort
RAM usage	Measures runtime memory requirement
Flash/model size	Measures storage requirement
Accuracy	Ensures optimization does not significantly degrade ML performance
Energy/inference*	Evaluates power efficiency where measurement facilities are available

*Energy measurement will depend on the available measurement setup.

The comparison will demonstrate whether the optimized framework provides measurable improvements over conventional MCU-based inference.

---

8. Expected Outcome

The expected outcome is a reusable TinyML execution framework capable of deploying selected lightweight ML models efficiently on the MCXN236.

The project is expected to demonstrate:

- Reduced inference latency compared with a baseline implementation.
- Efficient use of the Cortex-M33 computational capabilities.
- Reduced memory requirements through INT8 representation and optimized buffering.
- A common framework for multiple lightweight ML models.
- Automatic or semi-automatic selection of appropriate execution kernels.
- Preservation of acceptable model accuracy after quantization.
- Practical deployment of TinyML on a resource-constrained microcontroller.

A suitable embedded-system application will later be selected as a **case study** to validate the framework under a practical operating scenario.

---

9. Novelty and Significance

The primary focus of the project is not the development of a single ML model, but the development of a **configurable execution framework** that can adapt the implementation of different lightweight ML models to the capabilities and constraints of the MCXN236.

Instead of treating every ML application as an independent embedded implementation, the proposed framework provides a common structure for model conversion, configuration, optimized kernel selection, and inference. This makes the approach potentially reusable across different TinyML applications while remaining within the computational and memory limitations of a microcontroller.

---

10. Final Project Concept

In simple terms, the project will answer the following question:

> **Can we create a configurable software-based TinyML acceleration framework that automatically maps selected lightweight ML models to optimized execution kernels and improves their inference efficiency on the Cortex-M33-based MCXN236 microcontroller?**

The final system will therefore combine **machine learning, embedded C/C++ programming, ARM Cortex-M architecture, model quantization, optimization techniques, and performance evaluation** into a single TinyML framework.